

# RAHUL SHARMA

AUTOMATION ENGINEER | PLAYWRIGHT · PYTHON · PYTEST · CI/CD

Chandigarh, India | +91 78072 32349 | [fastasf48@gmail.com](mailto:fastasf48@gmail.com)

Portfolio: [fastasf48-hash.github.io](https://fastasf48-hash.github.io) | [github.com/fastasf48-hash](https://github.com/fastasf48-hash) | [linkedin.com/in/rahul-sharma-65093b1b9](https://linkedin.com/in/rahul-sharma-65093b1b9)

## PROFESSIONAL SUMMARY

Automation Engineer with **4+ years** in software quality assurance — one year in manual testing, then three-plus years designing and building test automation frameworks. Specialises in **Playwright with Python and Pytest**, covering UI, REST API and database layers in a single maintainable codebase wired into **Jenkins and GitHub Actions** CI/CD. Currently leading the migration of a legacy Selenium suite to Playwright for an enterprise airport access-control product, cutting execution time and eliminating the flaky failures that had eroded trust in the suite.

Focused on framework architecture: Page Object Model, deliberately scoped fixtures, parameterised data-driven tests, configuration-driven multi-environment execution, and reporting that makes test-run health visible to stakeholders outside QA. Four public, fully documented framework repositories — walkthroughs at [fastasf48-hash.github.io](https://fastasf48-hash.github.io).

## CORE SKILLS

**Test Automation:** Playwright, Pytest (fixtures, markers, parameterization, hooks, plugins), Page Object Model (POM), Selenium WebDriver, TestNG, data-driven and keyword-driven testing, cross-browser testing (Chromium, Firefox, WebKit), parallel execution (pytest-xdist)

**Languages:** Python, SQL, Java

**API Testing:** REST API automation, Playwright APIRequestContext, Requests, JSON Schema validation (jsonschema), Faker, pytest-playwright, Basic and Bearer token authentication

**CI/CD & DevOps:** Jenkins, GitHub Actions, Docker, Git, GitHub

**Databases:** Microsoft SQL Server, MySQL, PostgreSQL, SQLite, SQLAlchemy, parameterized queries, backend data-integrity validation

**Reporting & Logging:** Allure, pytest-html, Python logging, screenshot / video / trace capture on failure

**Enterprise Platforms:** Alert Enterprise Guardian (PIAM), Apache NiFi, Apache ZooKeeper, Apache Tomcat

**Practices:** SDLC, Agile / Scrum, OOP, Data Structures & Algorithms, test strategy, defect triage, code review, JIRA

## PROFESSIONAL EXPERIENCE

**Alert Enterprise** — Chandigarh, India

### Software Engineer II

Jul 2024 – Present

- Lead the migration of the legacy Selenium regression suite for **Airport Guardian** to a Playwright + Python + Pytest framework, improving execution speed and sharply reducing flaky failures.
- Architected the replacement framework around the **Page Object Model**, scoped Pytest fixtures and parameterised test data, so UI and API suites share one reusable, maintainable spine.
- Built REST API test coverage using **Requests, jsonschema and Faker** — request/response assertions, JSON Schema validation, and data-driven cases fed by dynamically generated test data.
- Integrated regression and API suites into **Jenkins CI/CD pipelines**, giving every merge request continuous testing and rapid feedback.
- Implemented **SQLAlchemy**-based database validation to confirm backend data integrity within end-to-end scenarios.
- Introduced structured **Allure and HTML reporting** plus centralised logging, making test-run health visible to cross-functional stakeholders instead of living in a console.
- Mentor QA engineers on Python, Pytest and Playwright fundamentals, accelerating the team's shift to modern automation practices.

### Software Engineer I

Jan 2022 – Jun 2024

- Achieved a **98% defect detection rate** on Airport Guardian through rigorous test design and execution, validating enterprise PIAM workflows built on Apache NiFi, Apache ZooKeeper, Tomcat and Microsoft SQL Server.
- Wrote Selenium (Java) automation for critical regression scenarios over approximately six months, cutting manual testing effort and test-cycle duration significantly — by **100% in several suites**.
- Streamlined testing processes in collaboration with cross-functional teams, shortening product release cycles.
- Partnered directly with customers to gather feedback and fold it into the QA process, increasing customer confidence.
- Ran QA process training for customer support teams, reducing customer-reported incidents by **30%**.

## SELECTED PROJECTS

**Playwright + Pytest Automation Framework** [github.com/fastasf48-hash/playwright-pytest-automation-framework](https://github.com/fastasf48-hash/playwright-pytest-automation-framework)

- UI, REST API and database suites in one repository: six page objects inheriting a shared **BasePage** that centralises error handling and logging, a full create/read/update/patch/delete lifecycle plus token auth on Playwright's APIRequestContext, and SQLite validation through a driver-agnostic wrapper using parameterized queries only.
- Multi-environment execution behind a **--env** flag, config-driven expected values, Excel- and Faker-sourced test data, JSON Schema response validation, and Allure / pytest-html reporting with screenshot, video and full trace retained on failure.

- Runs in parallel via pytest-xdist with group-aware scheduling across **Chromium, Firefox and WebKit**, from three CI targets: a GitHub Actions workflow, a Docker image, and a parameterised Jenkinsfile.

**AI Test Failure Analyzer for Playwright** [github.com/fastasf48-hash/AI-Test-Failure-Analyzer-For-Playwright](https://github.com/fastasf48-hash/AI-Test-Failure-Analyzer-For-Playwright)

- Captures the complete evidence bundle on every failure — screenshot, video, trace, console and network logs, stack trace — and triages it with a **free, deterministic rule-based classifier** that needs no API key, so the tool is useful on day one.
- Adds an opt-in LLM layer (OpenAI / Claude, pluggable provider) returning a **Pydantic-validated** root-cause report — category, confidence, severity, suggested fix and improved locator. Invalid model output never reaches the database, and the API can only be invoked from one auditable call site.
- Streamlit dashboard for execution summaries, failure trends and flaky-test tracking over a **SQLAlchemy 2.0** schema, linked to pytest-html and Allure reports, with a GitHub Actions pipeline that never spends an API call.

**Playwright Accessibility Toolkit** [github.com/fastasf48-hash/playwright-accessibility-toolkit](https://github.com/fastasf48-hash/playwright-accessibility-toolkit)

- Automated **WCAG 2.x** scanning built on axe-core: a one-line scan returns typed violation objects grouped by severity, and a threshold assertion fails the build once violations cross a configurable limit — dropping into any existing Playwright test.
- Filters by WCAG level through axe tags and is verified against both a clean and a deliberately inaccessible fixture page, proving it neither false-positives nor misses real violations.

**Selenium + TestNG Framework Template** [github.com/fastasf48-hash/Selenium-FrameWork](https://github.com/fastasf48-hash/Selenium-FrameWork)

- Reusable Java framework template built during my six months with Selenium: a **ThreadLocal WebDriver** factory making TestNG parallel execution safe by default, listener-driven ExtentReports with screenshot-on-failure and retry analysis, wait/element utilities, and Excel-backed TestNG DataProviders.

**EDUCATION**

|                                                                                    |                                    |
|------------------------------------------------------------------------------------|------------------------------------|
| <b>B.Tech, Computer Science</b> — Chandigarh Engineering College, Punjab           | 8.1 CGPA · 2022                    |
| <b>Senior Secondary &amp; Secondary (CBSE)</b> — DAV Dharamshala, Himachal Pradesh | Class XII 81.6% · Class X 9.2 CGPA |

**ACHIEVEMENTS**

- **Quality Champion** — awarded as a fresher for a 98% defect detection rate during my time as a manual tester.
- **Outstanding Performer** — awarded in two consecutive appraisal cycles for delivery on the Airport Guardian automation programme.
- **Team enablement** — mentored QA engineers onto Python, Pytest and Playwright; customer-support QA training reduced customer-reported incidents by 30%.